

Reminders

Upcoming due dates

Wed Pre-course survey

Fri Discussion Lab 1

#FinAid quiz on Canvas

Discussion section

Quiz 1: Intro/syllabus, Git lectures, and today

D1: Using git + installing Anaconda on your laptop

Data and data wrangling

Data Science in Practice

This week is still about giving you the tools

- Lecture is about data wrangling and pandas and what to (not) do with data
- D1 is out: Anaconda + GitHub
 - <https://git-scm.com/install/> to install git if you don't have it
 - <https://www.anaconda.com/download> if you don't have it
 - Get a GitHub login if you don't have it
- Go to section for help!
- See demo videos on Canvas if its helpful

The need to control versions

Or why D1 is asking you to use conda

- Not just your software (git) but also your entire environment affects how code works: OS, Python, libraries
- Three flavors of controlling the environment
 - Package managers (pip) let you install a given version, upgrade, downgrade, etc
 - Virtual environment managers (conda, venv) let you have different python+library environments and switch between them as you need for different projects
 - Virtual machines (docker, parallels) let you have different OS+python+library environments for different projects allowing you complete control and reproducibility

Use one of these

Data Structures

Structured data

- can be stored in database SQL
- tables with rows and columns
- requires a relational key
- 5-10% of all data

Semi-structured

- doesn't reside in a relational database
- but still has organizational properties
- CSV, XML, JSON

Unstructured

- non-tabular data
- 80% of the world's data by bytes
- No relationships inherent in the data
- images, text, audio, videos

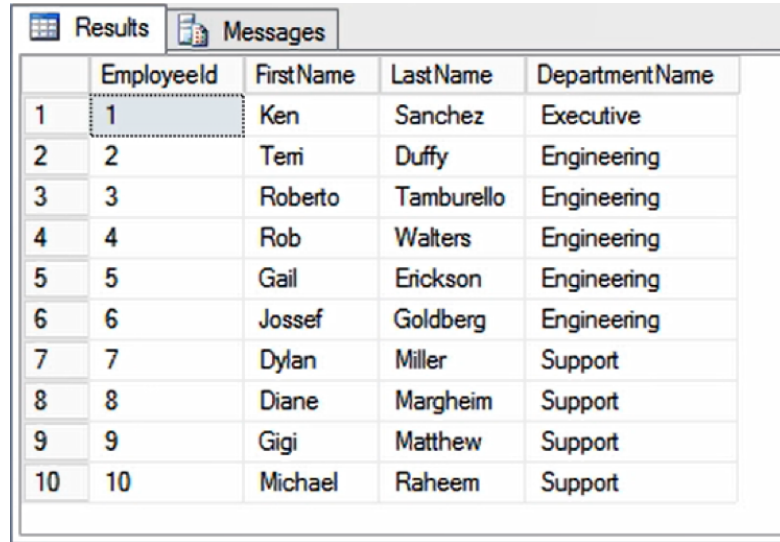
Structured Data

Databases! An organized collection of structured information

- Manage huge datasets
- Control access to data
- [Often] ensure consistency across huge number of users
- Allow users to find a subset of data with a query
- Run an analysis on data inside the DB and return a report

Relational Databases: A set of interdependent tables

1. Efficient Data Storage
2. Avoid Ambiguity
3. Increase Data Privacy

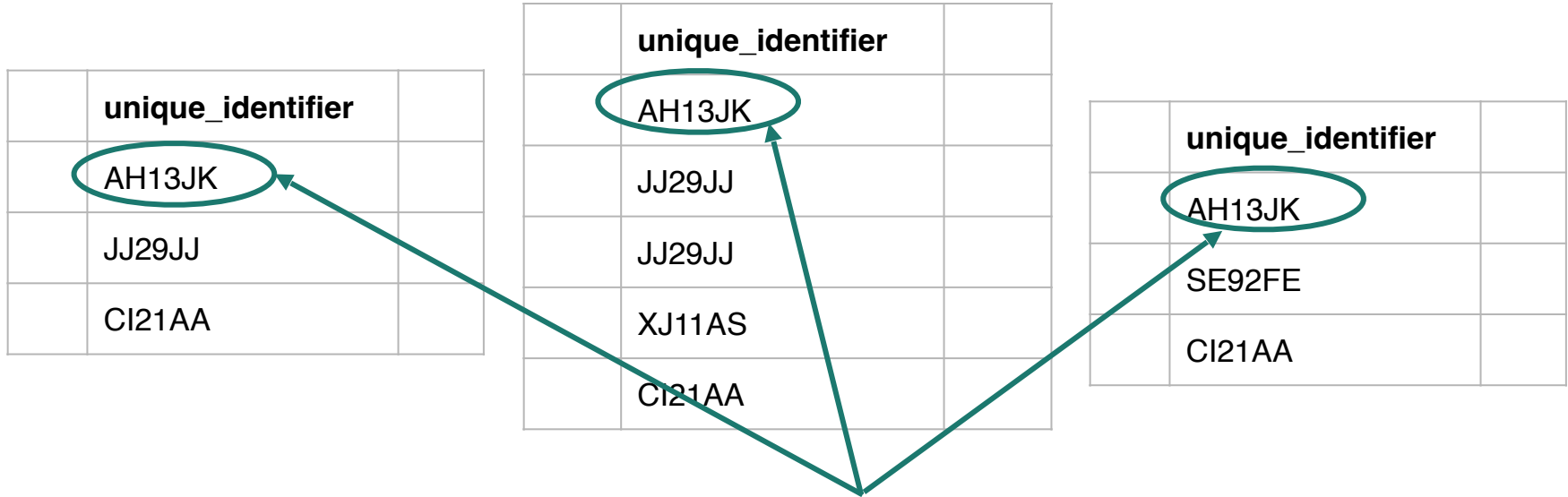


The image shows a screenshot of a database application window. At the top, there are two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with five columns: 'EmployeeId', 'FirstName', 'LastName', and 'DepartmentName'. The table contains 10 rows of data. The first row is highlighted with a blue background. The 'EmployeeId' column is also highlighted with a dotted border.

	EmployeeId	FirstName	LastName	DepartmentName
1	1	Ken	Sanchez	Executive
2	2	Teri	Duffy	Engineering
3	3	Roberto	Tamburello	Engineering
4	4	Rob	Walters	Engineering
5	5	Gail	Erickson	Engineering
6	6	Jossef	Goldberg	Engineering
7	7	Dylan	Miller	Support
8	8	Diane	Margheim	Support
9	9	Gigi	Matthew	Support
10	10	Michael	Raheem	Support

relational database

Information is stored across tables



entries are *related* to one another by their unique identifier

restaurant

name	id	address	type
Taco Stand	AH13JK	1 Main St.	Mexican
Pho Place	JJ29JJ	192 Street Rd.	Vietnamese
Taco Stand	XJ11AS	18 W. East St.	Fusion
Pizza Heaven	CI21AA	711 K Ave.	Italian

health inspections

id	inspection_date	inspector	score
AH13JK	2018-08-21	Sheila	97
JJ29JJ	2018-03-12	D'eonte	98
JJ29JJ	2018-01-02	Monica	66
XJ11AS	2018-12-16	Mark	43
CI21AA	2018-08-21	Anh	99

rating

id	stars
AH13JK	4.9
JJ29JJ	4.8
XJ11AS	4.2
CI21AA	4.7

restaurant

name	id	address	type
Taco Stand	AH13JK	1 Main St.	Mexican
Pho Place	JJ29JJ	192 Street Rd.	Vietnamese
Taco Stand	XJ11AS	18 W. East St.	Fusion
Pizza Heaven	CI21AA	711 K Ave.	Italian

health inspections

id	inspection_date	inspector	score
AH13JK	2018-08-21	Sheila	97
JJ29JJ	2018-03-12	D'eonte	98
JJ29JJ	2018-01-02	Monica	66
XJ11AS	2018-12-16	Mark	43
CI21AA	2018-08-21	Anh	99

rating

id	stars
AH13JK	4.9
JJ29JJ	4.8
XJ11AS	4.2
CI21AA	4.7

Two different restaurants with
the same name will have
different unique identifiers

Structured Data

Relational DB

- SQLite
- MySQL
- Postgres

Relational DB work using **tables of data** with “relationships” established between tables

Non-relational DB

- Hadoop
- Hive
- Apache CouchBase

NoSQL DB don't have tables but instead work with **key-value pairs** to lookup data

(Semi-)Structured Data

- Often tabular (rows and columns) like a spreadsheet or SQL DB
- But not always! There are other ways to organize data!
- Mainly: is formatted using a set of agreed-upon organization principles
- What we will mostly use

CSVs

Each column separated
by a comma



Example CSV - Sheet1 — Notatnik

Has the extension “.csv”

Plik Edycja Format Widok Pomoc

```
Email,First Name,Last Name,Company,Snippet 1  
example1@domain.com,John,Smith,Company 1,Snippet Sentence1  
example2@gmail.com,Mary,Blake,Company 2,Snippet Sentence 2  
example3@outlook.com,James,Joyce,Company 3,Snippet Sentence 3
```

Each row
is
separated
by a new
line



Example CSV



File Edit View Insert Format Data Tools Add-ons Help [All changes saved in Drive](#)

| 100% | \$ % .0 .00 123 | Arial | 10 | **B** *I* ~~ABC~~ A

fx

	A	B	C	D	E	F
1	Email	First Name	Last Name	Company	Snippet 1	
2	example1@domain.com	John	Smith	Company 1	Snippet Sentence1	
3	example2@gmail.com	Mary	Blake	Company 2	Snippet Sentence 2	
4	example3@outlook.com	James	Joyce	Company 3	Snippet Sentence 3	
5						
6						
7						
8						

CSV file



Example CSV - Sheet1 — Notatnik

Plik Edycja Format Widok Pomoc

Email,First Name,Last Name,Company,Snippet 1

example1@domain.com,John,Smith,Company 1,Snippet Sentence1

example2@gmail.com,Mary,Blake,Company 2,Snippet Sentence 2

example3@outlook.com,James,Joyce,Company 3,Snippet Sentence 3

JSON: key-value pairs

nested/hierarchical data

`{"Name": "Isabela"}`

key



value

JSON

These are all
nested within
attributes

```
"attributes": {  
  "Take-out": true,  
  "Wi-Fi": "free",  
  "Drive-Thru": true,  
  "Good For": {  
    "dessert": false,  
    "latenight": false,  
    "lunch": false,  
    "dinner": false,  
    "breakfast": false,  
    "brunch": false  
  },  
}
```

These are all
nested within
"Good For"

JSON

```
{
  "cells": [
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "This example represents the output the t-SNE dimensionality reduction algorithm on embeddings computed from Unicode emojis using Keras"
      ]
    },
    {
      "cell_type": "code",
      "execution_count": null,
      "metadata": {},
      "outputs": [],
      "source": [
        "import pandas as pd\n",
        "import holoviews as hv\n",
        "hv.extension('bokeh')\n"
      ]
    },
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "## Declaring data"
      ]
    },
    {
      "cell_type": "code",
      "execution_count": null,
```



is

{ j s o n }

Jupyter notebooks suck to version control

<https://nextjournal.com/schmudde/how-to-version-control-jupyter>

```
{  
  "cell_type": "code",  
  "execution_count": null,  
  "metadata": {},  
  "outputs": [],  
  "source": [  
    "import pandas as pd\n",  
    "import holoviews as hv\n",  
    "hv.extension('bokeh')"  
  ]  
},
```



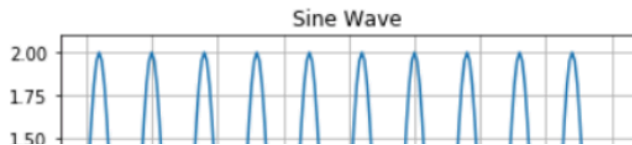
```
In [10]: import numpy as np
import matplotlib.pyplot as plt

# Data for plotting
t = np.arange(0.0, 2.0, 0.01)
s = 1 + np.sin((5 * 2)* np.pi * t)

# Note that using plt.subplots below is equivalent to using
# fig = plt.figure() and then ax = fig.add_subplot(111)
fig, ax = plt.subplots()
ax.plot(t, s)

ax.set(xlabel='time (s)', ylabel='voltage (mV)', title='Sine Wave')
ax.grid()
```

Out[10]:



"outputs": [

{

"data": {

"image/png":

"iVBORw0KGgoAAAANSUhEUgAAAYwAAAEWCAYAAAB1xKBvAAAABHNCSVQICAgIfAhkiAAAAA1wSFlzAAALEgAACxIB0t1+/AAAADl
 0RVh0U29mdHdhcmUAAbWF0cGxvdGxpYiB2ZXJzaW9uIDIuMi4yLCBodHRwOi8vbWF0cGxvdGxpYi5vcmcvhp/UCwAAIABJREFUeJz
 svXmcHNd13/s9vc4+2EgABHeQEKVSGGRFLembFNSPn7Wyy45i5UXh5ZjvcSy4xcr78WK5bkwzvkKSeIll0qaVxZKc0JLN+FHc0dx
 JEVxAAgQBAiCiIdbDP0tPT+80fVdXdmOnl1q17ezBm/T6f+QDdXVXnVtU996z3HFFKESNGjBgxYvRDYrkHECNGjBgxVgZigREjRow
 YMbQQC4wYMWLEiKGFWDGEiBEjRgwtxAIjRowYMWJoIRYYMWLEiBFDC7HAiBEDEJG/JiKPL/c4YsQ4nxELjBgfGojIXSLyoojMiMg

Jupyter notebooks suck to version control

<https://nextjournal.com/schmudde/how-to-version-control-jupyter>

Clear Output Manually

The simplest solution is to always clear the output before committing. **Cell** → **All Output** → **Clear** → **Save**. This removes any binary blobs that have been generated by the notebook. There are three main drawbacks:

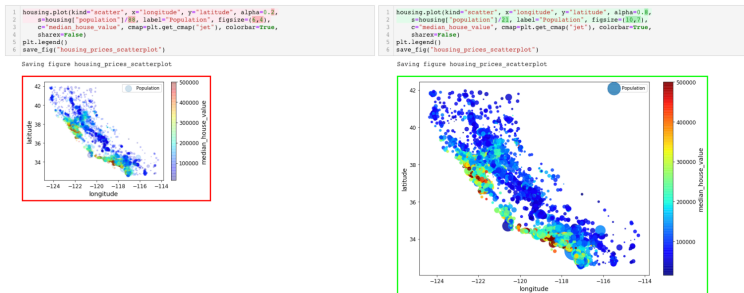
- It is a manual process.
- Collaborators on other machines will need to rerun the notebook to see the output, requiring additional time and setup.

Jupyter notebooks suck to version control

<https://nextjournal.com/schmudde/how-to-version-control-jupyter>

ReviewNB

ReviewNB is a GitHub app that also offers visual diffing with an interface that looks similar to the traditional Jupyter IDE. Because the outputs are visualized, problems associated with committing binary blobs disappear.



ReviewNB example courtesy of the [ReviewNB website](#)

The screenshot shows the ReviewNB interface, which is a web-based Jupyter IDE. The top bar includes menus for File, Edit, View, Run, Kernel, and Git. Below this is a sidebar with a file explorer showing 'Project.ipynb' and 'Terminal 2'. The main area displays the notebook content, including a scatter plot and a line plot. The scatter plot is titled 'Saving figure housing_prices_scatterplot' and the line plot is titled 'Low power AC'. The interface also shows a 'Changes' panel on the left, indicating that the notebook has been modified. The bottom right corner shows the 'Outputs' section, which includes a line plot titled 'Low power AC' showing a sine wave.

Back to data formats...

Extensible Markup Language (XML): nodes, tags, and elements

nested/hierarchical data

A node

\$node

<tag>

An opening tag

An element

<tag2> more content </tag2>

<tag3> more content </tag3>

</tag>

A closing tag

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<customers>
  <customer>
    <customer_id>1</customer_id>
    <first_name>John</first_name>
    <last_name>Doe</last_name>
    <email>john.doe@example.com</email>
  </customer>
  <customer>
    <customer_id>2</customer_id>
    <first_name>Sam</first_name>
    <last_name>Smith</last_name>
    <email>sam.smith@example.com</email>
  </customer>
  <customer>
    <customer_id>3</customer_id>
    <first_name>Jane</first_name>
    <last_name>Doe</last_name>
    <email>jane.doe@example.com</email>
  </customer>
</customers>
```

XML

Tidy Data

"Good data scientists understand, in a deep way, that the heavy lifting of cleanup and preparation isn't something that gets in the way of solving the problem: it is the problem."

- DJ Patil



Australian Bureau of Statistics

Table junk

1800.0 Australian Marriage Law Postal Survey, 2017

Released on 15 November 2017

Table 5 Participation by Federal Electoral Division(a), Males and Age Gender apartheid

Yeah NA

		18-19 years	20-24 years	25-29 years	30-34 years	35-39 years	40-44 years	45-49 years	50-54 years	55-59 years	60-64 years
Lingian(c)	Total participants	292	1,058	1,465	1,653	1,515	1,516	1,710	1,730	1,753	1,574
	Eligible participants	572	2,910	3,789	3,996	3,607	3,506	3,645	3,331	2,960	2,456
	Participation rate (%)	51.0	36.4	38.7	41.4	42.0	43.2	46.9	51.9	59.2	64.1
Solomon	Total participants	442	1,461	2,066	2,357	2,188	2,057	2,224	2,108	2,134	1,772
	Eligible participants	750	2,991	3,994	4,155	3,634	3,398	3,427	3,066	2,931	2,355
	Participation rate (%)	58.9	48.8	51.7	56.7	60.2	60.5	64.9	68.8	72.8	75.2
Northern Territory (Total)	Total participants	734	2,519	3,531	4,010	3,703	3,573	3,934	3,838	3,887	3,346
	Eligible participants	1,322	5,901	7,783	8,151	7,241	6,904	7,072	6,397	5,891	4,811
	Participation rate (%)	55.5	42.7	45.4	49.2	51.1	51.8	55.6	60.0	66.0	69.5
Australian Capital Territory Divisions	Total participants	1,764	4,789	4,817	4,973	4,626	4,453	5,074	4,826	5,169	4,394
	Eligible participants	2,260	6,471	6,448	6,509	5,983	5,805	6,302	5,902	6,044	5,057
	Participation rate (%)	78.1	74.0	74.7	76.4	77.3	76.7	80.5	81.8	85.5	86.9
Fenner(e)	Total participants	1,477	4,687	5,178	5,786	6,025	5,463	5,191	4,208	3,948	3,465
	Eligible participants	1,904	6,354	7,121	7,822	7,960	7,155	6,480	5,206	4,692	3,945
	Participation rate (%)	77.6	73.8	72.7	74.0	75.7	76.4	80.1	80.8	84.1	87.8
Australian Capital Territory (Total)	Total participants	3,241	9,476	9,993	10,739	10,651	9,916	10,263	9,634	9,117	7,639
	Eligible participants	4,164	12,825	13,569	14,331	13,943	12,960	12,782	11,108	10,736	9,002
	Participation rate (%)	77.8	73.9	73.7	75.1	76.4	76.5	80.3	81.3	84.9	87.3
Australia	Total participants	151,297	438,166	441,658	460,548	462,206	479,360	524,620	517,693	543,449	506,799
	Eligible participants	201,439	635,909	646,916	665,250	656,446	660,841	693,850	659,150	664,720	597,386
	Participation rate (%)	75.1	68.9	68.3	69.2	70.4	72.5	75.6	78.5	81.8	84.8

(a) The Federal Electoral Divisions are current as at 24 August 2017

(b) Includes those whose age is unknown

(c) Includes Christmas Island and the Cocos (Keeling) Islands

(d) Includes Norfolk Island

(e) Includes Jarvis Bay

Return of the table junk

1	area	gender	age	State	Area (sq km)	Eligible participants	Participation rate (%)	Total participants	Total Participants
2	Adelaide	Female	18-19 years	SA	76	1341	83.5	1120	1120
3	Adelaide	Female	20-24 years	SA	76	4620	81.2	3750	3750
4	Adelaide	Female	25-29 years	SA	76	4897	81.8	4004	4004
5	Adelaide	Female	30-34 years	SA	76	4784	79.8	3820	3820
6	Adelaide	Female	35-39 years	SA	76	4319	79	3411	3411
7	Adelaide	Female	40-44 years	SA	76	4310	80.6	3472	3472
8	Adelaide	Female	45-49 years	SA	76	4579	81.4	3728	3728
9	Adelaide	Female	50-54 years	SA	76	4475	84.7	3791	3791
10	Adelaide	Female	55-59 years	SA	76	4622	87.3	4033	4033
				SA	76	4342	89.3	3879	3879
				SA	76	3970	90.7	3602	3602
				SA	76	3009	90.3	2716	2716
				SA	76	2156	88.5	1908	1908
				SA	76	1673	85.1	1423	1423

 Australian Bureau of Statistics 1800.0 Australian Marriage Law Postal Survey, 2017 Released on 15 November 2017 Table 5 Participation by Federal Electoral Division(a), Males and Age Gender apartheid Table Junk									
Yearb NA									
Primary key(s)									
Merged cells									
Summary of data inside data									
Covariate as Subheading									
NA Yearb									
Return of the table Junk									
MS Excel or Doc									

urvey-data-with-r-5d35cea07962

Tidy Data

1. Each **variable** you measure should be in a single column

	A	B	C	D	E	F	G
1	ID	LastName	FirstName	Sex	City	State	Occupation
2	1004	Smith	Jane	female	Frederick	MD	Welder
3	4587	Nayef	Mohammed	male	Upper Darby	PA	Nurse
4	1727	Doe	Janice	female	San Diego	CA	Doctor
5	6879	Jordan	Alex	male	Birmingham	AL	Teacher

2. Every **observation** of a variable should be in a different row

	A	B	C	D	E	F	G
1	ID	LastName	FirstName	Sex	City	State	Occupation
2	1004	Smith	Jane	female	Frederick	MD	Welder
3	4587	Nayef	Mohammed	male	Upper Darby	PA	Nurse
4	1727	Doe	Janice	female	San Diego	CA	Doctor
5	6879	Jordan	Alex	male	Birmingham	AL	Teacher

3. There should be one table for each type of data

Demographic Survey Data

	A	B	C	D	E	F	G
1	ID	LastName	FirstName	Sex	City	State	Occupation
2	1004	Smith	Jane	female	Frederick	MD	Welder
3	4587	Nayef	Mohammed	male	Upper Darby	PA	Nurse
4	1727	Doe	Janice	female	San Diego	CA	Doctor
5	6879	Jordan	Alex	male	Birmingham	AL	Teacher

Doctor's Office Measurements Data

	A	D	E	F	G
1	ID	Height_inches	Weight_lbs	Insulin	Glucose
2	1004	65	180	0.60	163
3	4587	75	215	1.46	150
4	1727	62	124	0.72	177
5	6879	77	160	1.23	205

4. If you have multiple tables, they should include a column in each *with the same column label* that allows them to be joined or merged

	A	B	C	D	E	F	G
1	ID	LastName	FirstName	Sex	City	State	Occupation
2	1004	Smith	Jane	female	Frederick	MD	Welder
3	4587	Nayef	Mohammed	male	Upper Darby	PA	Nurse
4	1727	Doe	Janice	female	San Diego	CA	Doctor
5	6879	Jordan	Alex	male	Birmingham	AL	Teacher

	A	D	E	F	G
1	ID	Height_inches	Weight_lbs	Insulin	Glucose
2	1004	65	180	0.60	163
3	4587	75	215	1.46	150
4	1727	62	124	0.72	177
5	6879	77	160	1.23	205

Tidy data == rectangular data

A

	A	B	C	D	E
1	id	sex	glucose	insulin	triglyc
2	101	Male	134.1	0.60	273.4
3	102	Female	120.0	1.18	243.6
4	103	Male	124.8	1.23	297.6
5	104	Male	83.1	1.16	142.4
6	105	Male	105.2	0.73	215.7

Tidy Data Benefits

1. consistent data structure
2. foster tool development
3. require only a small set of tools to be learned
4. allow for datasets to be combined

TIDY data is **NOT** the same as **CLEAN** data

Clean Data

Data cleaning is the process of detecting and correcting/
removing data records that are

- incomplete,
- incorrect,
- inaccurate, or
- irrelevant

and often includes making sure missing data is correctly marked
and that duplicate records and other errors are removed

Data that is **TIDY** and **CLEAN** is ready for use

Tabular Data Time

<https://forms.gle/6EKypPu1odgBdbiJ8>



Clean

ID	Last	First	height_m	height_f
1004	Smith	Jane	NA	65
4587	Nayef	Mohammed	72	NA
1727	Doe	Janice	NA	60
6879	Jordan	Alex	55	NA

Tidy

ID	Last	First	sex	height
1004	Smith	Jane	female	65
4587	Nayef	Mohammed	male	72
1727	Doe	Janice	fem	60
6879	Jordan	Alex	male	55

Tidy + Clean

ID	Last	First	sex	height
1004	Smith	Jane	F	65
4587	Nayef	Mohammed	M	72
1727	Doe	Janice	F	60
6879	Jordan	Alex	M	55

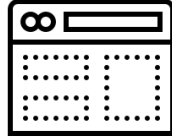
Unstructured Data

Some datasets record information about the state of the world, but in a more heterogeneous way. Perhaps it is a large text corpus with images and links like Wikipedia, or the complicated mix of notes and test results appearing in personal medical records.

Unstructured Data Types



Text files
and
documents



Websites
and
applications



Sensor
data



Image
files



Audio
files



Video
files



Email
data



Social
media
data



Positive:
70%

Negative:
20%

Neutral:
10%



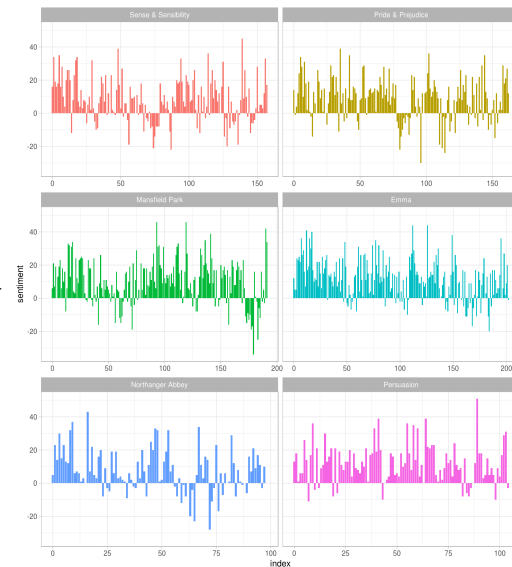
Text:
Sentiment Analysis



tidy dataset

Word	Novel	Frequency
good	Emma	359
young	Emma	192
friend	Emma	166

results



Application Programming Interface (API) access

How to get data from websites directly

- See this tutorial! <https://realpython.com/api-integration-in-python/>
- Basically:
 - Website gives you a way to authenticate (e.g. keys or OAuth)
 - You create a text string that asks for the thing you want in a particular way
 - You turn that text string into an HTTP request on the website
 - Website returns the thing you're looking for as XML or JSON or similar via HTTP

```
import requests

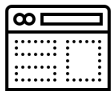
token = "YOUR_GITHUB_TOKEN"
headers = {
    "Authorization": f"Bearer {token}",
    "Accept": "application/vnd.github+json"
}

# Fetching all repositories for the authenticated user
response = requests.get("https://api.github.com/user/repos", headers=headers)

if response.status_code == 200:
    repos = response.json()
    for repo in repos:
        print(f"Repo Name: {repo['name']} - URL: {repo['html_url']}")
else:
    print(f"Failed: {response.status_code}")
```

Accessing data from a website

- Prefer API access
- If no API access then you can see if its possible to web scrape in a reasonable fashion
 - Be nice... don't slam the servers
 - Identify yourself as a scraper if at all possible



PYTHON

BEAUTIFULSOUP WEB SCRAPING

```
import requests
from bs4 import BeautifulSoup

def get_genius_lyrics(url):
    # User-Agent is required to avoid 403 Forbidden errors
    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36'
    }

    response = requests.get(url, headers=headers)
    if response.status_code != 200:
        return "Error: Could not reach page."

    # Parse HTML content
    soup = BeautifulSoup(response.content, 'lxml')

    # Genius uses different classes; this selector covers the most common ones
    lyric_selectors = ['div[class^="Lyrics__Container"]', '.song_body-lyrics p', 'div.lyrics']

    lyrics = []
    for selector in lyric_selectors:
        for tag in soup.select(selector):
            # strip=True removes leading/trailing whitespace; separator='\n' keeps line breaks
            t = tag.get_text(strip=True, separator='\n')
            if t:
                lyrics.append(t)

    return "\n".join(lyrics) if lyrics else "Lyrics not found."

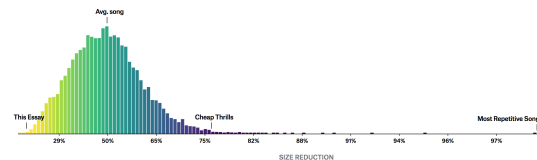
# Example Usage
song_url = "https://genius.com/Kendrick-lamar-not-like-us-lyrics"
print(get_genius_lyrics(song_url))
```

text (lyrics)

tidy dataset

song	Artist	Released	Reduction
Cheap Thrills	Sia	2016	76
Around The World	Daft Punk	1997	98
Everybody Dies	J. Cole	2018	27

results



Are Pop Lyrics Getting More Repetitive?